

MÓDULO III · SESIÓN 1

FUNDAMENTOS DE GIT:

Instalación, Configuración y Comandos Básicos

Programa de Especialización en Ingeniería de Datos
Universidad Ricardo Palma · IDIA

¿Qué veremos hoy?

Sesión de 3 horas — Módulo III / Sesión 1

01

Control de versiones

Qué es y por qué importa en datos

02

Git: arquitectura y áreas

Working dir, staging, repositorio

03

Instalación y configuración

git config, verificación inicial

04

Comandos esenciales

init, add, commit, status, diff, log

05

.gitignore para datos

Qué no versionar en proyectos Python

06

Push, Pull y Clone

Conectando con GitHub

08

Laboratorio práctico

Pipeline ETL con ventas.csv

¿Qué es el control de versiones?

VCS (VERSION CONTROL SYSTEM): *Software que registra y gestiona todos los cambios realizados en archivos a lo largo del tiempo.*



¿Qué cambios?

Qué líneas se modificaron en cada archivo



¿Por qué?

El mensaje del commit explica la razón



¿Quién?

Nombre y email del autor del cambio



¿Cuándo?

Fecha y hora exacta del commit

💡 En ingeniería de datos: versionar scripts de limpieza, pipelines ETL y configuraciones — no los datos crudos en sí.

Control de versiones

Título y resumen de
un dashboard

CAMBIO 1



CAMBIO 2



CAMBIO 3



Agrega un botón
para ver reporte

Agrega autores

```
1 <html lang="en">
2 <head>
3   <meta charset="UTF-8">
4   <meta name="viewport" content="width=device-width, initial-scale=1.0">
5   <title>Document</title>
6 </head>
7 <body>
8   <h1>Dashboard de Calidad de Datos</h1>
9   <h2>Resumen</h2>
10  <p>Esta página muestra un resumen sobre la calidad de los datos en diferentes fuentes utilizadas en el proyecto.</p>
11 </body>
12 </html>
13
```

```
1 <body>
2   <h1>Dashboard de Calidad de Datos</h1>
3   <h2>Resumen</h2>
4   <p>Esta página muestra un resumen sobre la calidad de los datos en diferentes fuentes utilizadas en el proyecto.</p>
5   <button>Ver reporte de calidad</button>
6 </body>
7 </html>
8
```

```
1 <body>
2   <h1>Dashboard de Calidad de Datos</h1>
3   <h2>Autores</h2>
4   <ul>
5     <li>Judith Quiñones</li>
6     <li>Ofelia Roque</li>
7   </ul>
8   <h2>Resumen</h2>
9   <p>Esta página muestra un resumen sobre la calidad de los datos en diferentes fuentes utilizadas en el proyecto.</p>
10  <button>Ver reporte de calidad</button>
11 </body>
```

Tipos de sistemas de control de versiones



Local

Solo en tu computadora

Ejemplo: RCS



Centralizado

Un servidor central; sin él, no puedes trabajar

Ejemplo: SVN, CVS



Distribuido ✓

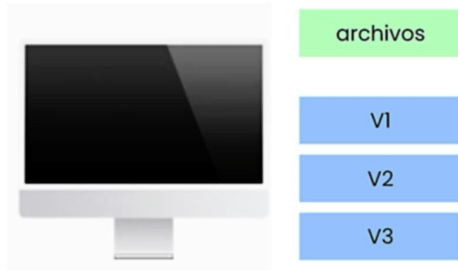
Cada computadora tiene el historial completo

Ejemplo: Git, Mercurial

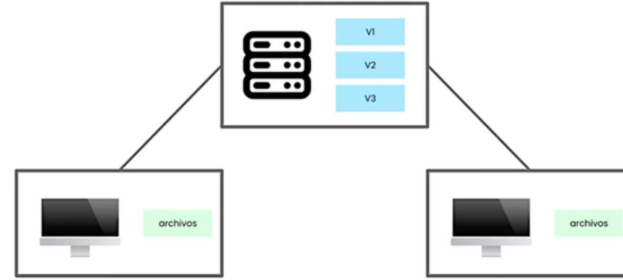


Tipos de sistemas de control de versiones

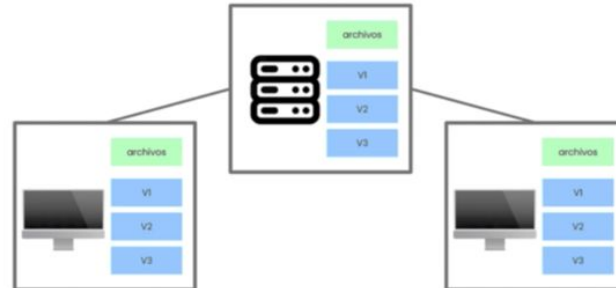
Control de versiones local



Control de versiones centralizado



Control de versiones distribuido



¿Qué es Git?

Sistema de control de versiones distribuido, gratuito y de código abierto, diseñado para gestionar proyectos de cualquier tamaño con velocidad y eficiencia.

Beneficio de Git	¿Por qué es útil?
Trabajo en equipo	Permite que varias personas colaboren en el mismo proyecto sin perder cambios.
Historial de cambios	Guarda quién, cuándo y qué cambió en cada archivo, y puedes regresar a versiones anteriores.
Ramas fáciles	Puedes probar nuevas ideas o solucionar errores sin afectar el trabajo principal.
Trabajo sin internet	Puedes hacer commits y avanzar sin conexión, y luego sincronizar con GitHub cuando tengas internet.
Seguridad e integridad	Cada cambio está protegido con un código único (SHA-1) que garantiza que no se altere nada sin dejar rastro.
Velocidad	Las operaciones locales (guardar cambios, crear ramas, etc.) son rápidas incluso en proyectos grandes.
Amplia compatibilidad	Funciona con muchos sistemas operativos, editores y plataformas en la nube (como GitHub o GitLab).

<https://git-scm.com/>



REPOSITORIO

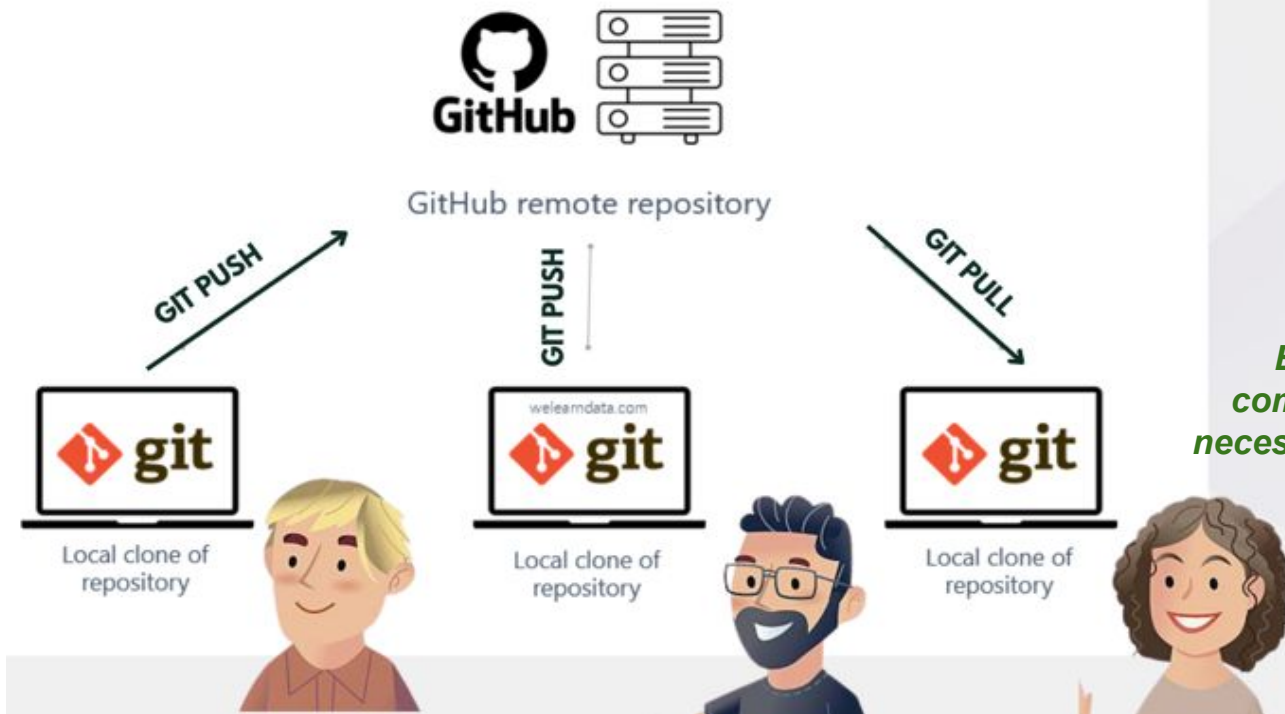
Espacio de memoria en el que se aloja el proyecto y su historial de versiones



Nombre	
	.env
	.env-example
> 	.git
	.gitignore
> 	node_modules
	nodemon.json
	package-lock.json
	package.json
	server.js
> 	src

REPOSITORIO

Es el que está en la nube o un servidor, por ejemplo en GitHub, GitLab, Bitbucket, etc.



En tu propia computadora, sin necesidad de internet.

Las 3 áreas de Git

Entender esto es la clave para no perderse

Working Directory

Área de trabajo

Archivos que estás editando ahora mismo. Git los ve pero aún no los rastrea.

— editas aquí —

Staging Area

Área de preparación

Cambios marcados para el próximo commit. Como un carrito de compras antes de pagar.

`git add`

Repository

Historial de versiones

Commits permanentes guardados en `.git/`. El historial oficial del proyecto.

`git commit`

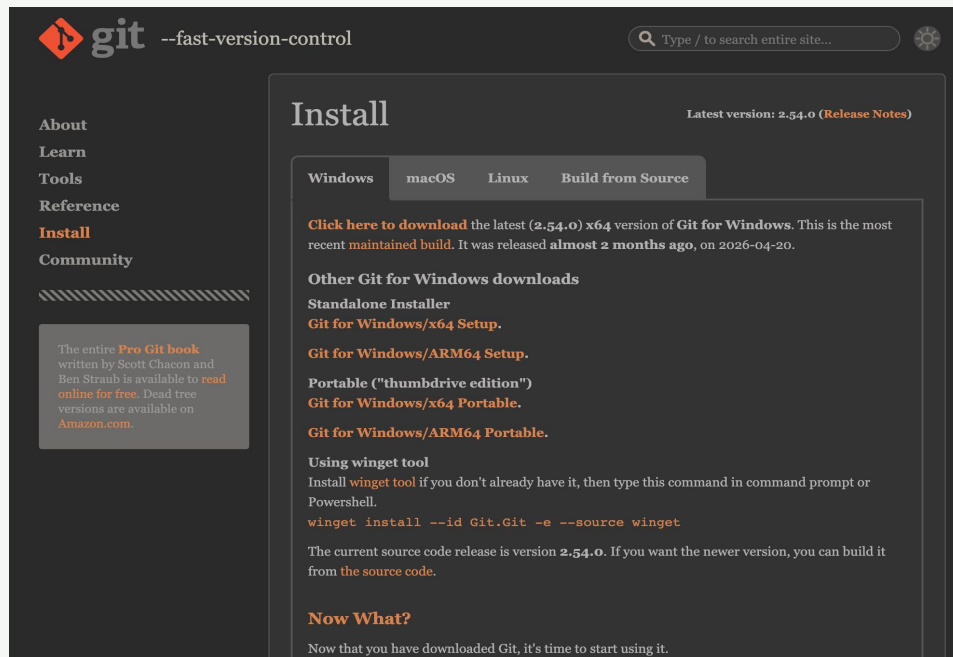
Instalación de Git

<https://git-scm.com/downloads>

[Cómo instalar Git en Windows y realizar la configuración inicial](#)

Verificar instalación:

```
terminal
$ git --version
git version 2.50.0
```



La terminal en Windows

Git instala su propia terminal — Git Bash

A diferencia de Mac/Linux que usan la terminal nativa, en Windows usarás Git Bash — viene incluida al instalar Git.

¿Cómo abrir Git Bash en Windows?

- 1 Busca 'Git Bash' en el menú Inicio y ábrelo
- 2 Clic derecho en cualquier carpeta → 'Git Bash Here'
- 3 Desde VS Code: Terminal → New Terminal (ya usa Git Bash automáticamente)

```
terminal — Git Bash
# Verificar que Git está listo
$ git --version
git version 2.50.0 ← deberías ver algo así
```

¿Qué es un IDE?

El lugar donde escribirás, ejecutarás y versionar tu código

IDE = Integrated Development Environment (Entorno de Desarrollo Integrado)



Editor de código con resaltado de sintaxis y autocompletado



Terminal integrada para correr comandos de Git y Python

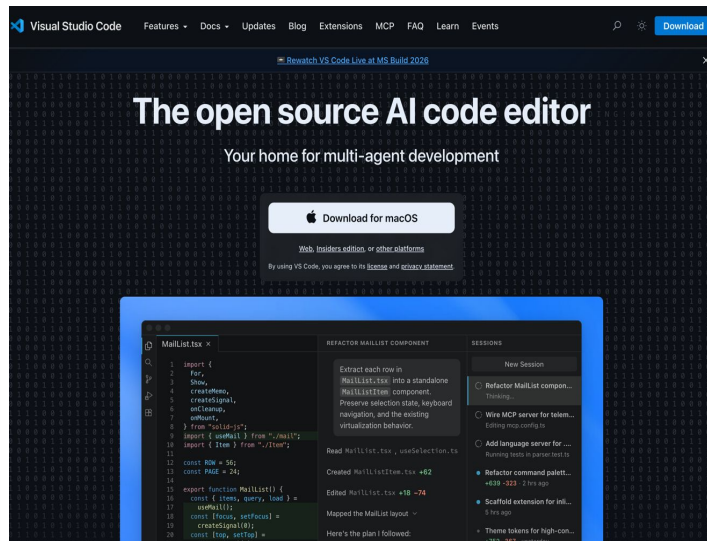


Depurador para encontrar y corregir errores



Extensiones: Git, Python, SQL, dbt y más

✓ Para este curso usaremos VS Code — gratuito, con terminal integrada y panel visual de Git. Cursor es una alternativa moderna con IA incorporada.



<https://code.visualstudio.com/>

[VISUAL STUDIO CODE: Tutorial para principiantes](#)

Configuración inicial

Solo se hace una vez por computadora

```
terminal
# Registrar tu identidad (aparecerá en cada commit)
$ git config --global user.name "Tu Nombre"
$ git config --global user.email "tu@email.com"

# Verificar configuración
$ git config --list
```

¿Por qué importa?



Identifica quién hizo cada cambio en el historial



Permite saber a quién preguntar sobre cada parte del código



--global aplica a todos los repositorios de tu computadora

git init · git status

Los primeros dos comandos que usarás

Genera un directorio llamado .git que incluye la configuración y el control de versiones - comando para iniciar

git init — Iniciar un repositorio

terminal

```
$ mkdir pipeline-ventas && cd pipeline-ventas  
$ git init  
Initialized empty Git repository in /pipeline-ventas/.git/
```

git status — Ver el estado actual

terminal

```
$ git status  
On branch main  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    scripts/limpiar_ventas.py  
    data/ventas.csv
```



Usa git status antes y después de cada comando. Es tu mejor aliado para no perderte.

git add — Mover al staging area

Prepara los cambios que quieres incluir en el commit

Comando	¿Qué hace?	Cuándo usarlo
<code>git add .</code>	Todo el directorio actual (más usado en el día a día)	★ Más común
<code>git add -A</code>	Todos los archivos del proyecto (nuevos, modificados, eliminados)	Proyecto completo
<code>git add scripts/limpiar.py</code>	Un archivo específico	Cambio puntual
<code>git add scripts/</code>	Todos los archivos de una carpeta	Por carpeta
<code>git add scripts/*.py</code>	Todos los .py dentro de scripts/	Por extensión
<code>git add -u</code>	Solo archivos modificados o eliminados (no los nuevos)	Sin nuevos archivos

git add — Patrones útiles en proyectos de datos

Variantes con wildcards (úsalas cuando sea necesario, no por defecto):

```
terminal
# Todos los .py en la carpeta scripts/
$ git add scripts/*.py

# Todos los .py incluyendo subcarpetas
$ git add scripts/**/*.py

# Varios archivos específicos a la vez
$ git add scripts/limpiar.py requirements.txt .gitignore

# Solo modificados/eliminados (NO agrega archivos nuevos)
$ git add -u
```

✓ En el 90% de los casos usarás
git add . y listo

⚠ Los wildcards (**/*.py) funcionan
mejor en Git Bash/Unix

git commit — Guardar en el historial

terminal

```
$ git commit -m "agrega limpieza de nulos en ventas.csv"
```

Buenas prácticas en mensajes de commit:

```
git commit -m "cambios"
```

✗ No dice qué cambió

```
git commit -m "fix"
```

✗ Demasiado vago

```
git commit -m "arreglé todo"
```

✗ No es específico

```
git commit -m "agrega limpieza de nulos en ventas.csv"
```

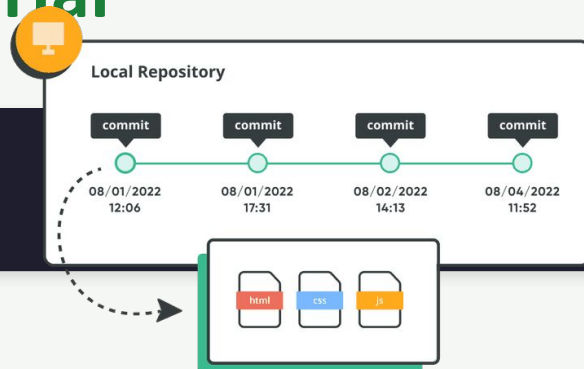
✓ Claro y específico

```
git commit -m "corrige tipo de dato en columna fecha"
```

✓ Explica el cambio

```
git commit -m "agrega columna total en transformacion"
```

✓ Verbo en presente



git diff · git log

Ver qué cambió y revisar el historial

git diff — Ver exactamente qué cambió

terminal — líneas en rojo = eliminadas · líneas en verde = agregadas

```
$ git diff
- df = df.dropna()
+ df = df.dropna(subset=['precio', 'cantidad'])
+ df = df[df['precio'] > 0]
```

git log --oneline — Historial compacto (más útil)

LISTA TODOS LOS COMMITS DE TODO EL PROYECTO

terminal

```
$ git log --oneline
a3f2c1d (HEAD -> main) agrega columna total
9b1e4fa corrige tipo de dato en columna fecha
5d8a2b3 agrega limpieza de nulos en ventas.csv
1c4f0e2 primer commit: estructura del proyecto
```

.gitignore — Qué NO versionar en datos

Equivalente a ignorar node_modules/ en frontend

Archivo .gitignore para proyectos Python / Ingeniería de Datos:

```
.gitignore
# — Datos crudos (pueden pesar GB) —
data/raw/
*.csv

# — Credenciales (;NUNCA en Git!) —
.env

# — Entorno Python —
venv/
__pycache__/
*.pyc

# — Notebooks con outputs —
.ipynb_checkpoints/
```

✓ Sí versionar

- scripts/*.py
- requirements.txt
- .env.example
- README.md
- data/processed/*.csv

✗ NO versionar

- .env (credenciales)
- data/raw/*.csv (datos crudos)
- venv/ (librerías)
- __pycache__/

Crear cuenta en GitHub

Solo necesitas un email — es gratuito

- 1 Ve a github.com y haz clic en Sign up
- 2 Ingresa tu email, crea una contraseña y elige un username
- 3 Verifica tu email con el código que te enviarán
- 4 Completa las preguntas de bienvenida (puedes saltarlas)
- 5 ¡Listo! Ya tienes tu repositorio en la nube



Usa el mismo email que registrarás en git config --global user.email para que tus commits se vinculen a tu cuenta.

Personal Access Token (PAT)

GitHub reemplazó la contraseña con un token de seguridad

Al hacer git push, GitHub pedirá tu username y un token — no tu contraseña de cuenta.

¿Cómo generar tu PAT? (Classic Token)

- ① GitHub → tu avatar (arriba derecha) → Settings
- ② Menú lateral → Developer settings → Personal access tokens → Tokens (classic)
- ③ Generate new token → elige expiración → marca el scope 'repo'
- ④ Haz clic en Generate token y ¡cópialo ahora! No lo verás de nuevo

 Guarda el token en un lugar seguro (ej. un bloc de notas temporal). Se usa como contraseña al hacer git push.

push · pull · clone

Conectando tu repo local con GitHub



git push

Sube tus commits locales al repositorio remoto en GitHub

```
git push origin main
```



git pull

Descarga y fusiona los cambios del remoto en tu copia local

```
git pull origin main
```



git clone

Copia un repositorio remoto completo a tu computadora

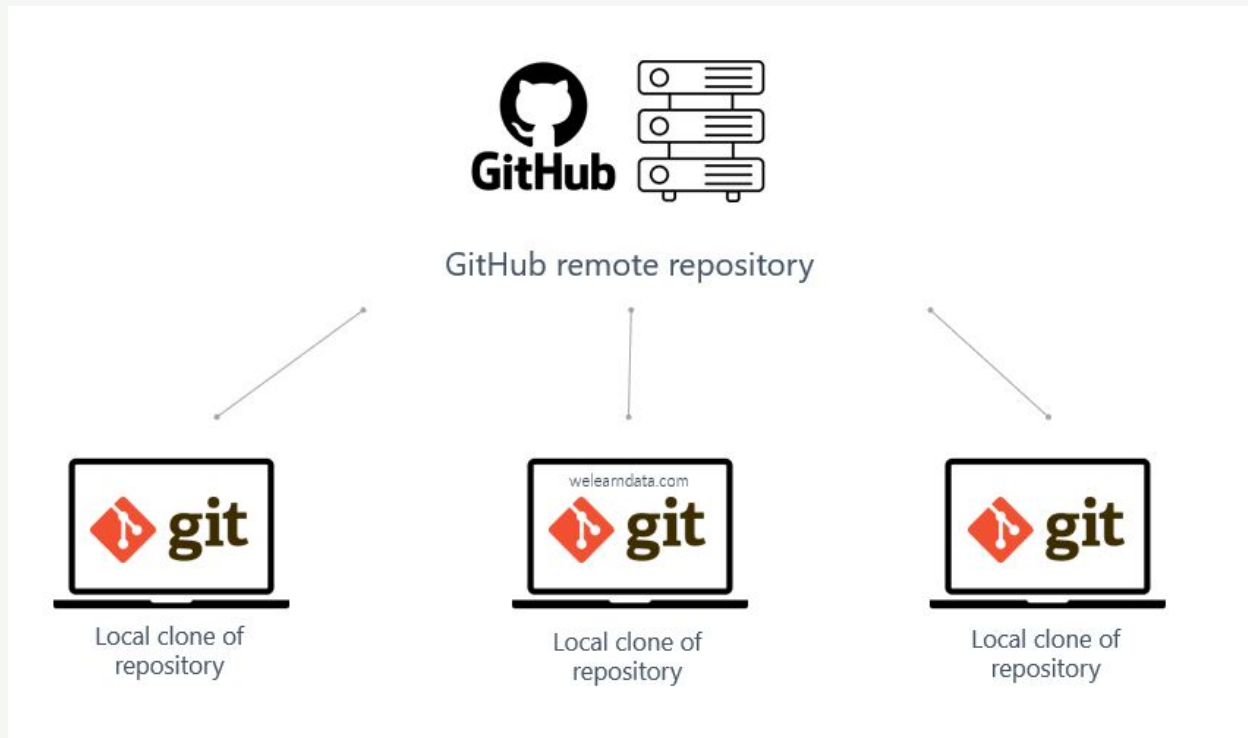
```
git clone https://github.com/user/repo.git
```



Flujo típico: `git add .` → `git commit -m '...'` → `git push origin main`

push · pull

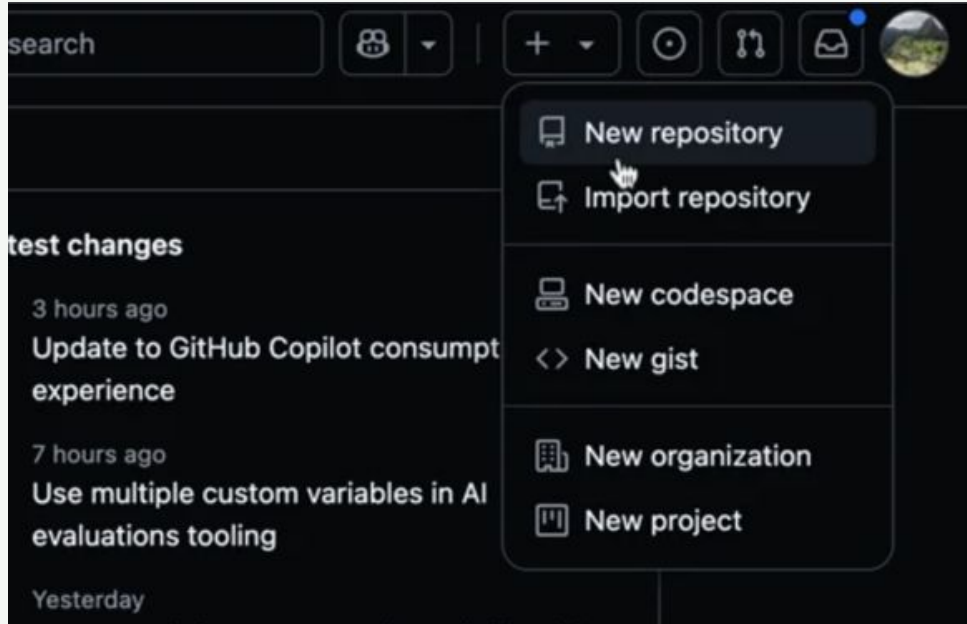
Conectando tu repo local con GitHub



REMOTO

push · pull

Conectando tu repo local con GitHub



CREAR NUEVO
REPOSITORIO

push · pull

Conectando tu repo local con GitHub

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

Owner *

 JudithCristina

Repository name *

 ejercicio-git is available.

Great repository names are short and memorable. Need inspiration? How about [expert-spork](#) ?

Description (optional)

☒ Public

Anyone on the Internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

☐ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs](#).

Add .gitignore

Choose which files not to track from a list of templates. [Learn more about ignoring files](#).

Choose a license

A license tells others what they can and can't do with your code. [Learn more about licenses](#).

 You are creating a public repository in your personal account.

Create repository

push · pull

Conectando tu repo local con GitHub

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

Owner *

 JudithCristina

Repository name *

 ejercicio-git is available.

Great repository names are short and memorable. Need inspiration? How about [expert-spork](#) ?

Description (optional)

☒ Public

Anyone on the Internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

☐ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs](#).

Add .gitignore

Choose which files not to track from a list of templates. [Learn more about ignoring files](#).

Choose a license

A license tells others what they can and can't do with your code. [Learn more about licenses](#).

 You are creating a public repository in your personal account.

Create repository

push · pull

Conectando tu repo local con GitHub

```
judithcristinaqi@Judith-local ejercicio_git % git remote add origin https://github.com/JudithCristina/ejercicio-git.git

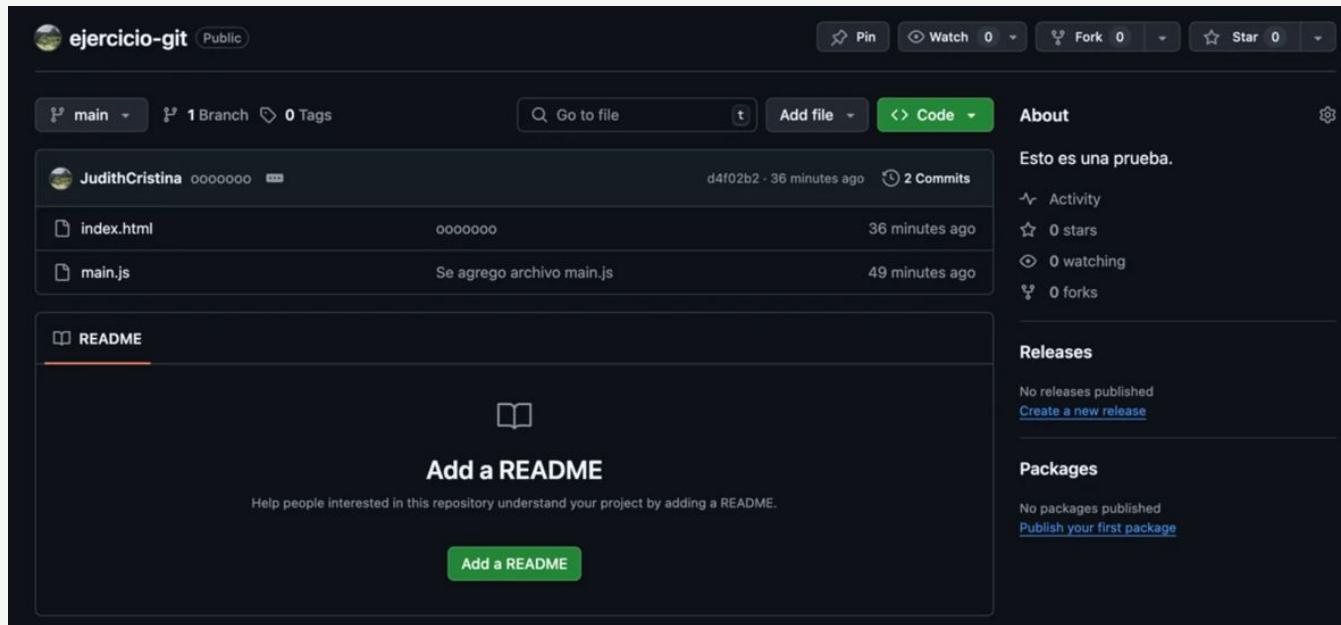
judithcristinaqi@Judith-local ejercicio_git % git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 10 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 507 bytes | 507.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/JudithCristina/ejercicio-git.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
judithcristinaqi@Judith-local ejercicio_git %
```

Te pedirá tu usuario de GitHub y tu Personal Access Token (PAT)

REMOTO

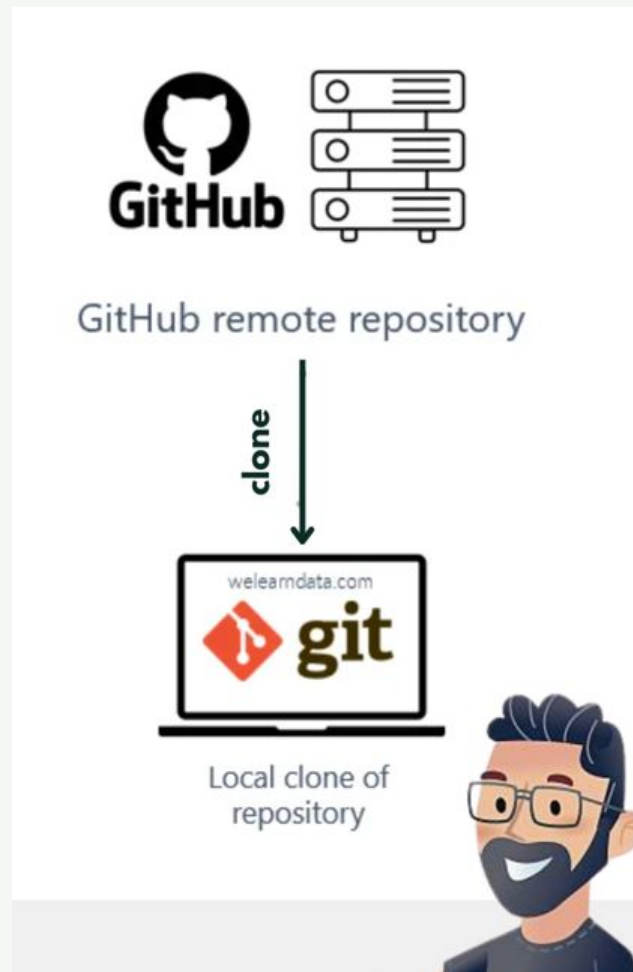
push · pull

Conectando tu repo local con GitHub



Repositorio subido a github

CLONE



REMOTO

CLONE

The screenshot shows the GitHub interface for the repository 'veterinary-registration' (Public). A notification banner at the top states 'Your master branch isn't protected' with options to 'Dismiss' or 'Protect this branch'. Below this, the repository is shown with the 'master' branch selected, 3 branches, and 0 tags. A search bar and 'Add file' button are visible. The file list on the left includes 'public', 'src', '.gitignore', 'README.md', 'debug.log', 'package-lock.json', and 'package.json'. The 'Code' button is highlighted in green. A 'Clone' dialog is open, showing the 'Local' and 'Codespaces' tabs. The 'Clone' button is highlighted with a red arrow. The dialog shows the 'HTTPS' tab selected, with the URL 'https://github.com/JudithCristina/veterinary-registration' and a 'Clone using the web URL' option. Other options include 'Open with GitHub Desktop' and 'Download ZIP'.

veterinary-registration Public

Dismiss Protect this branch

Your master branch isn't protected
Protect this branch from force pushing or deletion, or require status checks before merging. [View documentation](#).

master 3 Branches 0 Tags

Go to file t Add file <> Code

JudithCristina subiendo lista de pacientes

public	first commit
src	subiendo lista de p
.gitignore	first commit
README.md	first commit
debug.log	first commit
package-lock.json	subiendo lista de p
package.json	subiendo lista de pacientes 6 years ago

Clone

HTTPS SSH GitHub CLI

`https://github.com/JudithCristina/veterinary-registration`

Clone using the web URL.

Open with GitHub Desktop

Download ZIP

README

CLONE

```
judithcristinaqi@Judith-local ~ % cd veterinaria
judithcristinaqi@Judith-local veterinaria % git clone https://github.com/JudithCristina/veterinary-registration.git
Cloning into 'veterinary-registration'...
remote: Enumerating objects: 62, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 62 (delta 0), reused 0 (delta 0), pack-reused 58 (from 1)
Receiving objects: 100% (62/62), 598.09 KiB | 1.20 MiB/s, done.
Resolving deltas: 100% (11/11), done.
judithcristinaqi@Judith-local veterinaria %
```

```
judithcristinaqi@Judith-local veterinary-registration % ls
README.md                package-lock.json        public
debug.log                package.json             src
judithcristinaqi@Judith-local veterinary-registration % _
```


Revertir cambios

¿Cómo deshago lo que hice?

Hay 3 situaciones comunes — cada una tiene su comando:

1 ¿Modifiqué un archivo pero no hice git add todavía?

```
$ git restore archivo.py  
# Descarta los cambios y vuelve al último commit
```

2 ¿Hice git add pero aún no hice commit?

```
$ git restore --staged archivo.py  
# Saca el archivo del staging, pero conserva los cambios
```

3 ¿Ya hice commit y quiero deshacerlo de forma segura?

```
# Primero busca el hash: git log --oneline  
$ git revert a3f2c1d # crea commit que deshace los cambios
```

💡 git revert es seguro porque no borra el historial — solo añade un commit que revierte. Perfecto para repos compartidos.

Git vs GitHub

	Git	GitHub
¿Qué es?	Software de control de versiones	Plataforma web para alojar repos
¿Dónde corre?	En tu computadora (local)	En la nube (internet)
¿Necesita red?	No — funciona offline	Sí — es un servicio online
Colaboración	Local o en red interna	Colaboración global con PRs, issues
Comandos	git add, commit, push...	Crear repo, fork, pull request

Mapa de comandos de la sesión

```
git config --global user.name
```

Configurar tu nombre

```
git init
```

Iniciar repositorio

```
git status
```

Ver estado actual

```
git add .
```

Pasar todo al staging

```
git add archivo.py
```

Pasar un archivo al staging

```
git diff
```

Ver qué cambió exactamente

```
git commit -m "mensaje"
```

Guardar en el historial

```
git log --oneline
```

Ver historial compacto

```
git push origin main
```

Subir cambios a GitHub

```
git pull origin main
```

Bajar cambios de GitHub

```
git clone <url>
```

Copiar repo desde GitHub

¡Eso es todo por hoy!

Sesión 2 → Estrategias de ramas y flujos de trabajo

Sesión 3 → Pull requests, code review y buenas prácticas

Tarea para la próxima sesión:

Ahora crea TU propio repositorio desde cero, sin ayuda. Elige un tema que te interese o que uses en tu trabajo.

Lo único que necesitas:

- Un archivo con contenido real tuyo
- Mínimo 3 commits con mensajes descriptivos
- Súbelo a GitHub y comparte el link